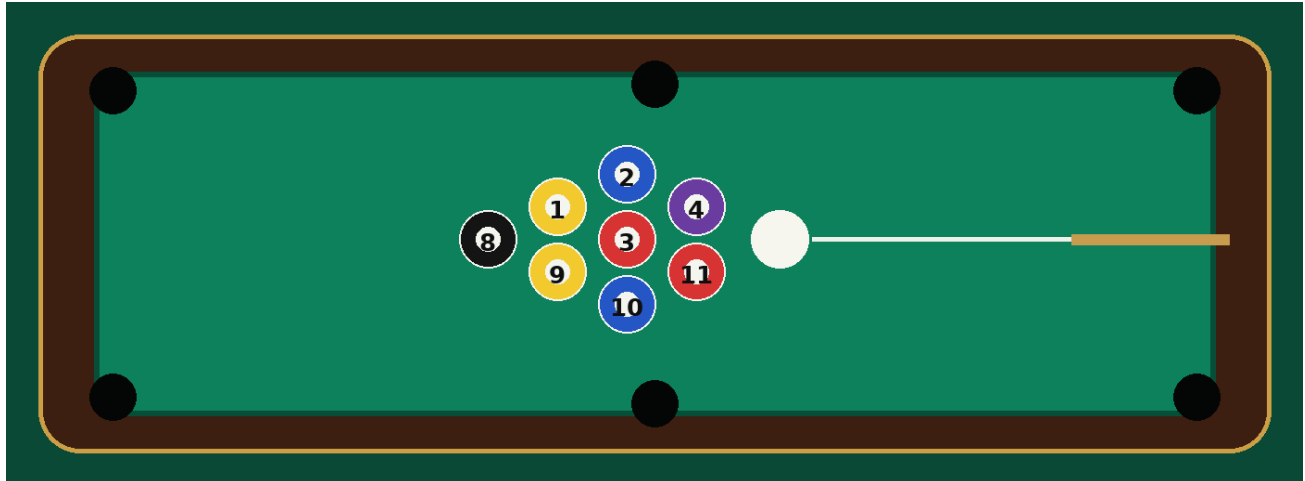


ARQUITETURA, LÓGICA, FÍSICA, IA, REGRAS E BANCO DE DADOS

Jogo de Sinuca

Tutorial Completo do Código



Jogo de Sinuca

8-Ball em ambiente 3D, com inteligência artificial, regras digitais, mira de precisão e ranking persistente.

Sumário

Os tópicos abaixo organizam o material em uma sequência progressiva, facilitando a consulta por assunto.

- 01. Arquitetura geral do projeto**
- 02. Mapa dos arquivos**
- 03. Ordem de carregamento no HTML**
04. Estrutura da interface em index.html
05. Sistema visual em styles.css
06. Responsividade
07. Estrutura principal de game.js
08. Constantes físicas e geometria da mesa
09. AudioEngine e efeitos sonoros
10. Classe Sinuca3D e estado central
11. Ligação dos controles e eventos
12. Inicialização da cena Three.js
13. Construção do ambiente e da mesa
14. Criação das bolas e texturas
15. Montagem do triângulo e reinício da mesa
16. Início de uma partida
17. Integração do jogo com o banco
18. Registro das estatísticas da partida
19. Entrada do jogador e validação da tacada
20. Criação do objeto de tacada
21. Loop físico a 360 Hz
22. Atrito, tabelas e movimento
23. Colisão entre bolas
24. Detecção de caçapas
25. Encerramento do movimento
26. Avaliação da saída
27. Avaliação das jogadas normais
28. Faltas e primeiro contato
29. Bola na mão
30. Mudança e manutenção do turno
31. Decisões especiais da saída
32. Inteligência Artificial
33. Dificuldade e ruído angular

34. Posicionamento da IA com bola na mão
35. Planejamento geométrico da IA
36. Verificação de caminhos bloqueados
37. Sistema de mira de precisão
38. Cálculo da bola fantasma e trajetórias
39. Sistema de câmeras
40. HUD e estado visual
41. Arquitetura de database.js
42. Criação do esquema SQLite
43. Persistência com IndexedDB e localStorage
44. Consultas e comandos SQL
45. Perfis e início de partidas
46. Registro de tacadas
47. Cálculo de pontos e encerramento transacional
48. Ranking e histórico
49. Exportação do SQLite
50. schema.sql
51. Loop de animação e inicialização final
52. Fluxo completo de uma jogada
53. Fluxo completo de uma partida
54. Pontos de manutenção e expansão

1. Arquitetura geral do projeto

O Jogo de Sinuca foi organizado como uma aplicação Web em camadas. O HTML define a interface; o CSS controla aparência e adaptação de tela; o JavaScript principal gerencia renderização 3D, física, regras, mira, câmeras e IA; um segundo JavaScript cuida do SQLite e da persistência; e a pasta banco documenta a estrutura relacional usada pelo sistema.

A escolha dessa divisão evita concentrar todas as responsabilidades em um único arquivo. A interface pode ser alterada sem reescrever a física; o banco pode evoluir sem mudar a renderização; e as regras ficam concentradas no controlador central da partida.

Arquivo	Linhas	Sistema	Finalidade
index.html	1-1526	Interface e carregamento	Contém HUD, mesa, controles, modais, ranking e referências aos scripts.
css/styles.css	1-1479	Apresentação	Define tema, painéis, botões, modais, HUD de bolas, ranking e responsividade.
js/game.js	1-21366	Motor do jogo	Three.js, física, regras, IA, mira, câmeras, som e fluxo da partida.
js/database.js	1-2224	Persistência	SQLite por sql.js, IndexedDB, localStorage, perfis, ranking e histórico.
banco/schema.sql	1-452	Modelo SQL	Criação das tabelas jogadores, partidas, tacadas e configuracoes.
INICIAR_JOGO.bat	arquivo curto	Inicialização no Windows	Atalho para abrir o jogo.

2. Mapa dos arquivos

O arquivo `index.html` é o ponto de entrada. Ele inclui `styles.css`, carrega `sql.js` e `Three.js` a partir de CDN, depois `database.js` e, por último, `game.js`. Essa ordem é importante: `game.js` espera que `THREE` e `SinucaDatabase` já estejam disponíveis quando a classe principal for criada.

`index.html` - referências principais próximas às linhas 65 e 1494-1518

```
<link rel="stylesheet" href="css/styles.css" />
<script src=".../sql-asm.js"></script>
<script src=".../three.min.js"></script>
<script src="js/database.js"></script>
<script src="js/game.js"></script>
```

3. Ordem de carregamento no HTML

1. O navegador interpreta a estrutura HTML e cria os elementos de interface.
2. `styles.css` é aplicado para montar o layout e o tema.
3. `sql.js` é carregado para disponibilizar SQLite em WebAssembly/JavaScript.
4. `Three.js` é carregado para disponibilizar cena, câmera, geometrias, materiais, luzes e renderizador WebGL.
5. `database.js` registra a classe `SinucaDatabase` no escopo global.
6. `game.js` cria o motor `Sinuca3D` e conecta a interface aos sistemas do jogo.

4. Estrutura da interface em `index.html`

O HTML usa identificadores para que `game.js` consiga localizar cada controle com `document.getElementById`. Em vez de montar os botões dinamicamente, a maior parte da interface já existe no documento e o JavaScript apenas atualiza conteúdo, classes e estados.

ID principal	Função na interface
<code>gameCanvas</code>	Canvas onde o <code>Three.js</code> desenha a cena.
<code>btnCamera</code>	Alterna os quatro modos de câmera.
<code>btnRanking</code>	Abre o modal de ranking e banco.
<code>btnSom</code>	Liga ou desliga os efeitos sonoros.
<code>btnRegras</code>	Abre as regras.
<code>powerRange</code>	Define a potência.
<code>btnTacada</code>	Dispara a jogada do lado humano.

ID principal	Função na interface
selectBola	Seleciona bola declarada.
selectCacapa	Seleciona cacapa declarada.
checkSafety	Marca a tacada de segurança.
gameLog	Recebe o histórico textual.
startModal	Modal inicial de perfil e dificuldade.
rankingModal	Modal de estatísticas e histórico.

5. Sistema visual em styles.css

styles.css define uma identidade visual escura com verde de mesa, detalhes dourados e painéis translúcidos. O CSS trata a interface como uma aplicação de jogo: barra superior, colunas laterais, viewport 3D, faixa de controle, cartões, modais e tabelas de ranking.

Arquivo	Linhas	Sistema	Finalidade
css/styles.css	3 em diante	Variáveis de tema	Cores e medidas reutilizadas.
css/styles.css	144 em diante	Barra superior	Alinhamento dos comandos e efeito translúcido.
css/styles.css	208-250	Botões	Estados normal, ghost, perigo, principal e TACADA.
css/styles.css	757 em diante	Modais	Tela inicial, regras e caixas de escolha.
css/styles.css	829 em diante	Propriedade das bolas	HUD de lisas/listradas e estado encaçapado.
css/styles.css	1062 em diante	Ranking	Cartões, tabela e histórico.

6. Responsividade

O layout se reorganiza por media queries. Em telas intermediárias, os painéis mudam de coluna; em telas menores, o painel esquerdo vira grade, a faixa de controle é simplificada e alguns cartões secundários são ocultados para priorizar a mesa.

Faixa	Comportamento
Até 1180 px	A grade principal reduz a largura da coluna lateral e reposiciona o painel direito.
Até 850 px	A estrutura passa para uma coluna principal e a mesa ganha prioridade.
Até 620 px	Botões e espaçamentos diminuem; controles passam a ocupar a largura disponível.
Até 560 px	Ranking e ações do banco se reorganizam para telas estreitas.

7. Estrutura principal de game.js

game.js é encapsulado por uma função autoexecutável e usa strict mode. Isso evita poluição desnecessária do escopo global. Dentro desse escopo são declaradas constantes físicas, funções auxiliares, AudioEngine e Sinuca3D.

game.js - início do arquivo

```
(() => {
  'use strict';
  const W = 5.0;
  const L = 10.0;
  const BALL_R = 0.145;
  ...
})();
```

8. Constantes físicas e geometria da mesa

As constantes W e L representam dimensões lógicas da área de jogo. BALL_R representa o raio das bolas. X_LIMIT e Z_LIMIT calculam os limites internos de movimento. STOP_SPEED define o ponto abaixo do qual uma bola é considerada parada. FRICTION, WALL_RESTITUTION e BALL_RESTITUTION controlam desaceleração e respostas de colisão.

Constante	Papel
W = 5.0	Largura lógica da superfície.
L = 10.0	Comprimento lógico.
BALL_R = 0.145	Raio usado na geometria e nas colisões.
STOP_SPEED = 0.035	Velocidade mínima antes de zerar o movimento.
FRICTION = 0.72	Decaimento exponencial de velocidade.
WALL_RESTITUTION = 0.88	Energia mantida em colisões com tabelas.
BALL_RESTITUTION = 0.965	Resposta de colisão bola-bola.

9. AudioEngine e efeitos sonoros

A classe AudioEngine começa em game.js, na linha 477. Ela usa Web Audio API em vez de arquivos de áudio externos. O método ensure() cria ou reativa o AudioContext. tone() gera tons; noise() gera ruído; e métodos específicos combinam esses recursos para representar taco, colisão, tabela, caçapa, falta, mudança de turno e vitória.

Arquivo	Linhas	Sistema	Finalidade
js/game.js	477-1267	AudioEngine	Motor de som.
js/game.js	986-1025	cue(power)	Som da tacada variando com potência.
js/game.js	1026-1093	collision(strength)	Som de choque entre bolas.
js/game.js	1094-1122	rail(strength)	Som de tabela.
js/game.js	1123-1160	pocket()	Som de encaçapamento.
js/game.js	1161-1198	foul()	Alerta de falta.

10. Classe Sinuca3D e estado central

Sinuca3D começa na linha 1261, com o construtor entre 1268 e 1818. O construtor cria referências para canvas, viewport, áudio, relógio, cena, câmera, renderizador, raycaster, bolas, taco, elementos de mira, estado da câmera, cronômetros, dificuldade, perfil e banco. Em seguida monta os sistemas principais e inicia o ciclo de renderização.

game.js - o objeto this.state é criado a partir da linha 1663 e concentra o estado da partida.

```
this.state = {
  started: false,
  moving: false,
  gameOver: false,
  turn: 'human',
  groups: { human: null, ai: null },
  breakShot: true,
  ballInHand: false,
  ballInHandHeadOnly: false,
  ballPlacementValid: true,
  shot: null
};
```

Esse objeto state funciona como a verdade central da partida. A interface, a IA, as regras e o controle de entrada consultam esse estado antes de permitir qualquer ação.

11. Ligação dos controles e eventos

bindUI(), linhas 1942-3306, reúne referências dos elementos e registra eventos de clique, teclado, mouse e redimensionamento. O botão de câmera percorre quatro posições. O botão de tacada chama userShoot(). O canvas recebe eventos de ponteiro usados tanto na mira quanto no posicionamento da branca.

Arquivo	Linhas	Sistema	Finalidade
js/game.js	1942-3306	bindUI()	Conecta HTML ao motor.
js/game.js	11037-11200	onPointerMove()	Muda a mira ou a posição provisória da branca.
js/game.js	11201-11356	onPointerDown()	Confirma interações do ponteiro.
js/game.js	11357-11455	pointerToTable()	Converte ponteiro da tela para coordenadas da mesa.

12. Inicialização da cena Three.js

init3D(), linhas 3307-3776, cria Scene, fog, PerspectiveCamera e WebGLRenderer. O renderizador usa antialiasing, sombras, encoding sRGB e tone mapping ACES. A iluminação combina HemisphereLight, SpotLight e PointLight para produzir a aparência do salão de sinuca.

game.js - init3D(), linhas 3307-3776

```
this.scene = new THREE.Scene();
this.camera = new THREE.PerspectiveCamera(44, 1, 0.1, 100);
this.renderer = new THREE.WebGLRenderer({
  canvas: this.canvas, antialias: true, powerPreference: 'high-performance'
});
```

A razão de pixel é limitada a 2 para equilibrar nitidez e desempenho. Isso evita que telas de densidade muito alta multipliquem excessivamente o custo de renderização.

13. Construção do ambiente e da mesa

createRoomDetails(), linhas 3777-3993, adiciona detalhes do ambiente. createTable(), linhas 3994-5110, monta base, madeira, pano, tabelas laterais, caçapas, diamantes, pernas, linha de cabeça e um plano invisível usado para converter o ponteiro em coordenadas da mesa.

O plano invisível é especialmente importante porque o mouse existe em coordenadas 2D de tela. O raycaster do Three.js projeta um raio pela câmera e encontra o ponto de interseção com esse plano, permitindo transformar o movimento do mouse em mira sobre a superfície 3D.

14. Criação das bolas e texturas

createBallTexture(), linhas 5111-5403, desenha cada textura em um canvas 2D. Para bolas listradas, cria uma faixa colorida sobre fundo claro; para bolas lisas, usa a cor integral. Os marcadores numéricos tornam as bolas legíveis em diferentes rotações.

createBalls(), linhas 5404-5828, usa SphereGeometry e MeshPhysicalMaterial. A bola branca recebe material claro sem textura. As bolas objeto recebem as texturas geradas. Cada bola é registrada em balls e ballById com posição, velocidade, estado ativo e referência visual.

15. Montagem do triângulo e reinício da mesa

rackBalls(), linhas 6562-6697, posiciona as quinze bolas em formação inicial. newRack(), linhas 6698-6951, reinicia estados da mesa, grupos, turno, bola na mão e outros controles associados a uma abertura. Esses métodos reutilizam os objetos 3D já existentes.

16. Início de uma partida

startGame(), linhas 6952-7175, lê o nome, dificuldade e estado do banco; prepara o perfil; reinicia estatísticas; monta a mesa; fecha o modal inicial; ajusta o HUD e inicia o registro da partida. A

divisão entre `newRack()` e `startGame()` separa a preparação física da mesa do contexto completo da partida.

17. Integração do jogo com o banco

`initDatabase()`, linhas 7246-7408, inicializa `SinucaDatabase`. Se `sql.js` não estiver disponível, o jogo continua sem persistência. Quando o banco está pronto, a rotina carrega perfis e o resumo do nome digitado, mantendo o motor independente da camada de dados.

Arquivo	Linhas	Sistema	Finalidade
js/game.js	7246-7408	<code>initDatabase()</code>	Inicializa a camada SQLite.
js/game.js	7409-7467	<code>refreshPlayerList()</code>	Preenche o datalist de perfis.
js/game.js	7468-7608	<code>refreshProfilePreview()</code>	Mostra o resumo do perfil antes da partida.
js/game.js	7664-7796	<code>beginTrackedMatch()</code>	Cria o registro inicial da partida.

18. Registro das estatísticas da partida

`emptyMatchStats()`, linhas 7176-7245, prepara os contadores. `recordShotStats()`, linhas 8047-8377, transforma a tacada interna em um registro compacto: número, autor, declaração, potência, ângulo, primeiro contato, bolas encaçapadas, falta, motivos, scratch, segurança e duração.

`finalizeTrackedMatch()`, linhas 8378-8586, envia ao banco vencedor, dificuldade, duração, grupos e estatísticas agregadas. Depois consulta o perfil para refletir os números no HUD e registra no histórico a variação de pontos.

19. Entrada do jogador e validação da tacada

`userShoot()`, linhas 11694-12053, é a porta de entrada da tacada humana. Antes de iniciar a física, verifica se a partida começou, se é o turno humano, se as bolas estão paradas, se a partida não terminou e se não existe bola na mão aguardando colocação.

A rotina lê a opção de segurança, a bola escolhida e a caçapa. Em declarações manuais comuns, uma bola diferente da 8 exige caçapa e também precisa pertencer aos alvos legais. Quando todas as bolas do grupo já saíram da mesa e a bola 8 é o único alvo, o seletor passa para a 8 e nenhuma caçapa é exigida.

game.js - `userShoot()`, linhas 11694-12053

```

if (!this.state.started || this.state.turn !== 'human' ||
    this.state.moving || this.state.gameOver || this.state.ballInHand) return;

// Declarações manuais comuns exigem caçapa para bolas diferentes da 8.
// Quando a bola 8 é o único alvo legal, a caçapa não é obrigatória.

```

20. Criação do objeto de tacada

startShot(), linhas 12054-12640, converte ângulo e potência em velocidade da bola branca. A fórmula usa seno no eixo X e cosseno no eixo Z, criando o vetor inicial que alimenta a simulação física.

game.js - startShot(), linhas 12054-12640

```

const dirX = Math.sin(angle);
const dirZ = Math.cos(angle);
const speed = 3.0 + clamp(power, 5, 100) * 0.175;
cue.vx = dirX * speed;
cue.vz = dirZ * speed;

```

A mesma rotina cria state.shot. Esse objeto é fundamental porque acompanha o que ocorreu durante o movimento: primeiro contato, bolas encaçapadas, scratch, contatos com tabela, bolas fora, grupo anterior, estado da mesa, potência, ângulo e instante de início.

21. Loop físico a 360 Hz

O loop visual usa requestAnimationFrame, mas a física não depende diretamente da taxa de quadros. animate(), linhas 21125-21366, acumula o tempo decorrido e executa physicsStep() em passos fixos de 1/360 de segundo. Isso reduz diferenças causadas por computadores com velocidades de renderização distintas e melhora a estabilidade de colisões rápidas.

game.js - o passo fixo de 1/360 aparece nas linhas 21167-21181 dentro de animate().

```

const step = 1 / 360;
while (this.accumulator >= step) {
  this.physicsStep(step);
  this.accumulator -= step;
}

```

22. Atrito, tabelas e movimento

physicsStep(), linhas 12641-13381, percorre as bolas ativas, integra posição, aplica decaimento exponencial de velocidade, trata tabelas, caçapas, bolas fora da mesa e solicita as colisões entre bolas.

game.js - physicsStep(), linhas 12641-13381

```
b.x += b.vx * dt;
b.z += b.vz * dt;
const decay = Math.exp(-FRICTION * dt);
b.vx *= decay;
b.vz *= decay;
if (speed < STOP_SPEED) b.vx = b.vz = 0;
```

As tabelas são tratadas por limites X e Z. Quando uma bola ultrapassa um limite fora da boca de uma caçapa, a posição é corrigida e o componente correspondente da velocidade troca de sinal, multiplicado por WALL_RESTITUTION. O impacto também alimenta o som de tabela e o registro de que houve contato com tabela após o primeiro choque.

23. Colisão entre bolas

resolveBallCollision(), linhas 13382-13963, detecta sobreposição pela distância entre centros e pelo diâmetro mínimo. Antes de resolver o impulso, tenta recuar matematicamente as bolas até o instante aproximado do primeiro contato dentro do passo físico. Isso reduz erro angular em contatos rápidos e cortes finos.

Depois do recuo, calcula o vetor normal da colisão, corrige a sobreposição restante e projeta a velocidade relativa nessa normal. Quando as bolas estão se aproximando, aplica um impulso multiplicado por BALL_RESTITUTION. Como ambas têm massa equivalente no modelo, o impulso é transferido de maneira simétrica.

game.js - resolveBallCollision(), linhas 13382-13963

```
const rel = (a.vx - b.vx) * nx + (a.vz - b.vz) * nz;
if (rel > 0) {
  const impulse = rel * BALL_RESTITUTION;
  a.vx -= impulse * nx;
  a.vz -= impulse * nz;
  b.vx += impulse * nx;
  b.vz += impulse * nz;
}
```

Essa função também registra firstContact quando uma das bolas envolvidas é a branca. Esse dado depois será usado pelas regras para saber se o primeiro alvo tocado era permitido.

24. Detecção de caçapas

detectPocket(), linhas 13964-14036, mede a distância da bola até cada centro de caçapa. pocketBall(), linhas 14127-14348, desativa a bola, zera sua velocidade, oculta a malha e registra o evento dentro da tacada em andamento.

Se a bola é a branca, state.shot.scratch recebe true. Se é bola objeto, o identificador é adicionado a pocketed e pocketedDetailed, incluindo a caçapa. Essa diferença permite avaliar regras de scratch, grupo e declaração depois que o movimento terminar.

25. Encerramento do movimento

`allStopped()` verifica se todas as bolas inativas ou ativas estão abaixo de `STOP_SPEED`.

`finishShotIfSettled()` exige que o estado de parada permaneça por mais de 0,24 segundo. Esse pequeno atraso evita encerrar uma jogada durante uma oscilação numérica muito curta. Só então zera velocidades restantes e chama `evaluateShot()`.

26. Avaliação da saída

`evaluateShot()`, linhas 14481-14560, direciona a jogada para `evaluateBreak()` quando `breakShot` está ativo. `evaluateBreak()`, linhas 14561-15132, verifica bola 8, quantidade de bolas encaçapadas, contatos com tabela, `scratch` e bolas fora.

`game.js` - `evaluateBreak()`, linhas 14561-15132

```
const illegalSpread = shot.pocketed.length === 0 && shot.breakRailBalls.size < 4;
const foul = shot.scratch || shot.offTableBalls.length > 0;
```

A bola 8 na saída abre uma decisão específica. Uma abertura sem encaçapamento e com menos de quatro bolas objeto alcançando tabela é marcada como saída ilegal. Em uma saída legal com bola objeto encaçapada, a primeira bola objeto encaçapada define imediatamente o grupo de quem executou a saída e o grupo oposto para o adversário.

27. Avaliação das jogadas normais

`evaluateNormalShot()`, linhas 15133-16285, concentra a maior parte das regras do 8-Ball. Primeiro identifica quem atirou, o oponente e o grupo atual. Depois monta a lista de faltas com base em `scratch`, bolas fora, primeiro contato e necessidade de encaçapamento ou contato posterior com tabela.

Em mesa aberta, a bola 8 não pode ser o primeiro contato enquanto ainda houver bolas dos grupos disponíveis. Com grupos definidos, o primeiro contato deve ser do grupo correto; depois que o grupo está limpo, o alvo esperado passa a ser a bola 8.

A função trata a bola 8 antes da continuidade comum. Se a 8 caiu, verifica se ela já era um alvo legal e se houve falta. Se o grupo estava completamente limpo antes da tacada e não houve falta, o lado que executou a jogada vence. Não existe exigência de caçapa declarada para a bola 8.

28. Faltas e primeiro contato

A regra de primeira bola atingida nasce na física e é consumida na avaliação. `resolveBallCollision()` grava o primeiro contato da branca. `evaluateNormalShot()` compara esse número com o alvo esperado. Além disso, a física marca `railAfterContact` quando qualquer bola toca uma tabela depois do primeiro contato. Assim, uma jogada sem encaçapamento ainda pode ser válida se houver contato posterior com tabela.

`game.js` - `evaluateNormalShot()`, linhas 15133-16285

```
if (shot.firstContact !== null && !anyObjectPocketed && !shot.railAfterContact) {
  fouls.push('nenhuma bola foi encaçada e nenhuma bola atingiu tabela após o primeiro contato');
}
```

29. Bola na mão

giveBallInHand(), linhas 17082-17296, ativa o estado ballInHand e pode limitar a colocação à região de cabeça. findFreeCuePosition(), linhas 17297-17385, procura uma posição segura de reserva. pointerToTable(), previewCuePlacement() e onPointerDown() cooperam para a escolha visual da posição da branca.

isPositionFree(), linhas 17022-17081, verifica se o ponto proposto não se sobrepõe a outra bola. Esse teste impede configurações fisicamente inválidas.

30. Mudança e manutenção do turno

keepTurn(), linhas 17386-17481, e passTurn(), linhas 17482-17576, centralizam a troca de responsabilidade. As rotinas mudam state.turn, HUD, áudio e, quando necessário, agendam a IA.

31. Decisões especiais da saída

breakEightDecision(), breakEightFoulDecision(), illegalBreakDecision() e breakFoulDecision(), entre as linhas 16286 e 16778, criam os fluxos especiais da saída. showChoices(), linhas 16779-16938, monta o modal com título, texto e botões de ação.

spotEight() recoloca a bola 8 em posição válida. Esse isolamento é importante porque recolocar uma bola exige verificar ocupação e evitar sobreposição com outras bolas.

32. Inteligência Artificial

A IA começa por queueAI(), linhas 17577-17636, que agenda a jogada apenas quando o estado permite. playAI(), linhas 17637-17995, trata bola na mão, saída e jogada normal. Em jogadas normais, solicita um plano geométrico a findBestAIShot().

33. Dificuldade e ruído angular

difficultyNoise(), linhas 17996-18051, adiciona erro aleatório ao ângulo. No Fácil, a faixa é aproximadamente $\pm 0,105$ radiano; no Médio, $\pm 0,032$; no Avançado, $\pm 0,009$. Assim, a precisão muda sem exigir três motores completamente separados.

game.js - difficultyNoise(), linhas 17996-18051

```
if (this.selectedDifficulty === 'facil') return rand(-0.105, 0.105);
if (this.selectedDifficulty === 'medio') return rand(-0.032, 0.032);
return rand(-0.009, 0.009);
```

34. Posicionamento da IA com bola na mão

`chooseAIPlacement()`, linhas 18052-18247, testa diversas posições. Cada candidata precisa respeitar os limites, a região de cabeça quando aplicável e a ausência de sobreposição. A pontuação favorece uma posição com boa proximidade a algum alvo legal.

Se nenhuma posição testada superar as restrições, a IA usa `findFreeCuePosition()` como alternativa segura.

35. Planejamento geométrico da IA

`findBestAIShot()`, linhas 18248-18975, combina cada alvo legal com as seis caçapas. Para cada par, calcula a direção da bola alvo até a caçapa e cria um ponto ghost, isto é, a posição em que o centro da branca deveria chegar para transferir impulso na direção desejada.

`game.js` - o ponto ghost usado pela IA aparece próximo da linha 18394 dentro de `findBestAIShot()`.

```
const ghost = {
  x: target.x - ux * BALL_R * 2.03,
  z: target.z - uz * BALL_R * 2.03
};
```

Depois a IA rejeita candidatos cujo ponto ghost está fora da mesa, cujo caminho branca->ghost está bloqueado ou cujo caminho alvo->caçapa está bloqueado. Também calcula distância, ângulo de corte e penalidade de borda. A função `score` combina esses fatores e ordena as opções.

A dificuldade decide como usar essa lista: Fácil pode escolher entre várias candidatas boas; Médio restringe a escolha às primeiras; Avançado usa a melhor. A potência ideal é estimada com distância da branca, distância à caçapa e ângulo de corte, depois recebe uma variação dependente da dificuldade.

36. Verificação de caminhos bloqueados

`segmentClear()`, linhas 18976-19137, testa se alguma bola ativa está próxima demais do segmento entre dois pontos. O método projeta cada bola sobre o segmento, limita a projeção ao intervalo válido e mede a distância até o ponto projetado. Se a distância for pequena demais, o caminho é considerado bloqueado.

37. Sistema de mira de precisão

`updateAimVisuals()`, linhas 19324-20418, ajusta taco, linha branca, bola fantasma, marcador de impacto, seta da bola alvo e linha azul da branca. A rotina só exibe esses elementos no turno humano, com as bolas paradas e sem bola na mão.

Primeiro, `rayToRail()` calcula até onde um raio na direção da mira alcançaria uma tabela. Em seguida, `findAimFirstBall()` encontra a primeira bola cujo círculo de colisão intercepta esse raio. Se houver colisão prevista, a mira usa a posição calculada do primeiro contato.

38. Cálculo da bola fantasma e trajetórias

Quando existe um alvo, `updateAimVisuals()` calcula a normal do contato e estima a velocidade de entrada com a mesma relação potência->velocidade usada em `startShot()`. A velocidade projetada sobre a normal gera um impulso aproximado para a bola alvo. Essa componente define a seta amarela.

`game.js` - a decomposição da velocidade prevista aparece próxima das linhas 19965-20040 dentro de `updateAimVisuals()`.

```
const relativeNormalSpeed = Math.max(0, inVX * nx + inVZ * nz);
const impulse = relativeNormalSpeed * BALL_RESTITUTION;
const targetVX = impulse * nx;
const targetVZ = impulse * nz;
```

A linha azul é calculada subtraindo da velocidade da branca a parte transferida na normal. O resultado representa a componente inicial que tende a permanecer na branca depois do primeiro choque. É uma previsão do primeiro evento; colisões posteriores não são simuladas pela mira.

39. Sistema de câmeras

`cameraMode` varia de 0 a 3. `updateCameraTarget()`, linhas 20821-21124, define posições alvo diferentes para 3D, Superior, Jogador e Arena lateral. No modo Jogador, a câmera é calculada a partir da bola branca e da direção da mira.

Modo	Posição aproximada definida no código
3D	(6.25, 6.25, 8.35), olhando para o centro da mesa.
Superior	(0, 14.15, 0.01), olhando verticalmente para a superfície.
Jogador	Atrás da branca, 3.35 unidades na direção oposta à mira, altura 2.22.
Arena lateral	(9.55, 10.55, 0.04), enquadramento elevado lateral.

`animate()` usa `lerp` para interpolar a posição e o ponto de olhar, criando transição suave entre câmeras. `updateRoomVisibilityForCamera()` oculta elementos do teto em visões em que poderiam bloquear a mesa.

40. HUD e estado visual

`updateUI()`, linhas 9195-9715, concentra grande parte das mudanças visuais derivadas do estado: nome, turnos, dificuldade, grupos, pontuação do perfil, controles habilitados e mensagens. `updateOwnershipHUD()`, linhas 9716-10254, desenha a propriedade das bolas e diferencia bolas ativas e já encaçapadas.

`getRemainingCounts()` conta lisas, listradas e bola 8 ainda ativas. `getLegalTargets()` usa grupos e contagens para devolver quais bolas podem ser atingidas por um lado. `updateCallOptions()` usa essa lista para atualizar os seletores de declaração.

41. Arquitetura de database.js

`database.js` encapsula SQLite na classe `SinucaDatabase`, iniciada na linha 179. O jogo chama métodos como `ensurePlayer()`, `startMatch()`, `logShot()`, `finishMatch()`, `getRanking()` e `getRecentMatches()` em vez de espalhar SQL pelo motor da partida.

42. Criação do esquema SQLite

`init()`, linhas 230-337, carrega `sql.js`, tenta recuperar um arquivo binário salvo, cria `Database` a partir dele ou inicia um banco vazio, chama `createSchema()` e persiste o resultado. `createSchema()`, linhas 338-426, usa `CREATE TABLE IF NOT EXISTS` para permitir inicialização repetida.

Tabela	Responsabilidade
jogadores	Perfil e estatísticas acumuladas.
partidas	Resumo de cada confronto.
tacadas	Detalhes de cada jogada.
configuracoes	Pares chave/valor para opções persistentes.

43. Persistência com IndexedDB e localStorage

O SQLite usado por `sql.js` vive em memória. Para persistir o banco entre sessões, `database.js` exporta o arquivo binário. `openIDB()`, linhas 427-547, abre o IndexedDB. `loadBinary()`, linhas 548-759, tenta recuperar o binário; `persist()`, linhas 760-959, grava novamente os dados, com `localStorage` como alternativa.

IndexedDB é a preferência porque suporta dados binários maiores com mais eficiência. `localStorage` atua como alternativa, convertendo bytes para Base64. Se ambos falharem, o jogo ainda mantém o SQLite apenas em memória durante a sessão.

44. Consultas e comandos SQL

`all()`, linhas 960-1046, `one()`, linhas 1047-1080, e `run()`, linhas 1081-1118, centralizam execução de SQL e parâmetros. `all()` retorna várias linhas; `one()` devolve a primeira; `run()` executa comandos de alteração e recupera o identificador quando necessário.

45. Perfis e início de partidas

`ensurePlayer()`, linhas 1119-1218, normaliza o nome, procura um perfil existente sem diferenciar maiúsculas de minúsculas e cria o registro quando necessário. `listPlayers()`, linhas 1219-1241, alimenta a lista de perfis. `getPlayerSummary()`, linhas 1242-1328, calcula também o aproveitamento.

`startMatch()`, linhas 1329-1390, insere uma partida com jogador, horário inicial e dificuldade. Os campos de resultado são completados no encerramento.

46. Registro de tacadas

`logShot()`, linhas 1391-1552, grava os dados principais da tacada: número, autor, bola e caçapa declaradas quando existirem, potência, ângulo, primeiro contato, bolas encaçapadas, falta, motivos, scratch, segurança, duração e horário.

database.js - `logShot()`, linhas 1391-1552

```
INSERT INTO tacadas (
  partida_id, numero, jogador, bola_declarada, cacapa_declarada, potencia, angulo,
  primeiro_contato, bolas_embocadas, falta, motivos_falta, branca_embocada,
  seguranca, duracao_ms, criada_em
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

47. Cálculo de pontos e encerramento transacional

`pointsForResult()`, linhas 1553-1604, define +10/+20/+35 em vitórias e -5/-8/-10 em derrotas.

`finishMatch()`, linhas 1605-2019, limita os pontos a no mínimo zero, calcula sequências e grava o resultado da partida e as estatísticas do jogador.

As duas atualizações são executadas dentro de `BEGIN TRANSACTION`, `COMMIT` e `ROLLBACK`. Isso foi feito para que o resumo da partida e o perfil sejam alterados como uma única unidade: se ocorrer um erro no meio, o banco reverte o conjunto em vez de ficar parcialmente atualizado.

database.js - `finishMatch()`, linhas 1605-2019

```
this.db.run('BEGIN TRANSACTION');
try {
  ... UPDATE partidas ...
  ... UPDATE jogadores ...
  this.db.run('COMMIT');
} catch (err) {
  this.db.run('ROLLBACK');
  throw err;
}
```

48. Ranking e histórico

`getRanking()`, linhas 2020-2057, ordena por pontos, vitórias, aproveitamento e nome.
`getRecentMatches()`, linhas 2058-2091, usa JOIN entre partidas e jogadores para devolver os confrontos concluídos do perfil em ordem decrescente.

`database.js` - `getRanking()`, linhas 2020-2057

```
ORDER BY pontos_ranking DESC, vitorias DESC, aproveitamento DESC, nome COLLATE NOCASE
```

49. Exportação do SQLite

`exportDatabase()`, linhas 2092-2224, exporta o conteúdo de `this.db` para bytes, cria um Blob e dispara o salvamento de `sinuca3d.sqlite`. Esse recurso oferece uma cópia portátil do banco persistido pelo navegador.

50. schema.sql

`banco/schema.sql` repete a estrutura relacional em formato SQL independente. O arquivo possui as tabelas jogadores, partidas, tacadas e configuracoes, com chaves, valores padrão e relacionamentos. Ele funciona como documentação executável do modelo de dados e facilita abrir a estrutura em uma ferramenta SQLite externa.

Arquivo	Linhas	Sistema	Finalidade
<code>banco/schema.sql</code>	10-132	jogadores	Perfil e estatísticas acumuladas.
<code>banco/schema.sql</code>	133-290	partidas	Resumo de cada partida.
<code>banco/schema.sql</code>	291-432	tacadas	Registro detalhado de jogadas.
<code>banco/schema.sql</code>	433-452	configuracoes	Configurações em chave/valor.

51. Loop de animação e inicialização final

`animate()`, linhas 21125-21366, chama `requestAnimationFrame` para manter o ciclo contínuo. Além da física, controla o término da tacada, a câmera, elementos de mira, visibilidade do ambiente e

renderização. Nas linhas finais do arquivo, DOMContentLoaded cria window.sinuca3D = new Sinuca3D().

game.js - inicialização final a partir da linha 21291

```

window.addEventListener('DOMContentLoaded', () => {
  try {
    window.sinuca3D = new Sinuca3D();
  } catch (err) {
    ...
  }
});

```

52. Fluxo completo de uma jogada

1. O lado humano move o mouse; onPointerMove() atualiza aimAngle.
2. updateAimVisuals() calcula linha, bola fantasma e previsões.
3. userShoot() valida turno e declarações comuns; quando a bola 8 é o único alvo, ela não exige caçapa.
4. startShot() converte ângulo e potência em velocidade e cria state.shot.
5. animate() executa physicsStep() em passos fixos de 1/360 s.
6. physicsStep() move bolas, aplica atrito, tabelas, caçapas e colisões.
7. resolveBallCollision() registra o primeiro contato quando a branca toca uma bola objeto.
8. pocketBall() registra scratch ou encaçapamentos.
9. finishShotIfSettled() espera todas as bolas pararem.
10. evaluateShot() aplica regras da saída ou regras normais.
11. keepTurn(), passTurn(), giveBallInHand() ou endGame() definem o próximo estado.
12. recordShotStats() envia os dados da jogada para o SQLite.

53. Fluxo completo de uma partida

1. startGame() lê perfil e dificuldade.
2. ensurePlayer() cria ou recupera o perfil.
3. beginTrackedMatch() cria a linha da partida no banco.
4. newRack() prepara a mesa e ativa a saída.
5. As jogadas alternam entre controle humano e playAI().
6. Cada tacada gera dados físicos, de regras e de banco.

7. `endGame()`, linhas 19138-19323, define o vencedor, bloqueia novas tacadas, grava o encerramento e apresenta a mensagem final. O modal de fim de partida oferece somente o botão Sair, que retorna à tela inicial.
8. `finalizeTrackedMatch()` envia o resumo ao banco.
9. `finishMatch()` grava partida, perfil, sequência e pontos dentro de uma transação.
10. O HUD e o modal de ranking consultam novamente os dados persistidos para refletir o resultado.

54. Pontos de manutenção e expansão

A separação atual permite evoluir o jogo por partes. Alterações de visual podem ser concentradas em CSS; novas estatísticas entram em `database.js` e no esquema; novos modos de câmera podem ser adicionados em `updateCameraTarget()`; mudanças de IA entram em `findBestAIShot()`; e ajustes de física ficam nas constantes e em `physicsStep()/resolveBallCollision()`.

Objetivo de alteração	Ponto principal
Mudar aparência da mesa	<code>createTable()</code> e materiais <code>Three.js</code> .
Adicionar câmera	<code>bindUI()</code> e <code>updateCameraTarget()</code> .
Ajustar precisão da IA	<code>difficultyNoise()</code> e seleção de candidatos.
Mudar estratégia da IA	<code>findBestAIShot()</code> e função de <code>score</code> .
Ajustar atrito	<code>FRICTION</code> e <code>physicsStep()</code> .
Ajustar elasticidade bola-bola	<code>BALL_RESTITUTION</code> e <code>resolveBallCollision()</code> .
Criar nova estatística	<code>emptyMatchStats()</code> , <code>recordShotStats()</code> , <code>database.js</code> e <code>schema.sql</code> .
Mudar ranking	<code>pointsForResult()</code> e <code>getRanking()</code> .
Alterar regra da saída	<code>evaluateBreak()</code> e métodos de decisão.
Alterar regra de jogada normal	<code>evaluateNormalShot()</code> .
Alterar mira	<code>updateAimVisuals()</code> , <code>rayToRail()</code> e <code>findAimFirstBall()</code> .

Visão final da arquitetura

O núcleo usa estado central, passo físico determinístico, regras avaliadas após cada tacada, IA baseada em geometria e SQLite isolado. Essa separação mantém renderização, regras, estratégia e persistência independentes e coordenadas.